

Exercice 1 : P et NP

En informatique, la conjecture $P = NP$ fait partie des plus importants problèmes non résolus ce jour.

En vous appuyant sur les éléments vus en cours, répondez aux questions suivantes, sous la forme d'un texte d'une quinzaine de lignes, argumentant vos propos. Votre texte rédigé permettra d'adresser la problématique $P = NP$ et de répondre entre autres aux questions suivantes :

1. Quand on évoque $P = NP$, que signifie qu'un problème $p \in P$?
2. Quand on évoque $P = NP$, que signifie qu'un problème $p \in NP$?
3. Donner des exemples de problèmes de P et de NP .
4. Que signifie le problème $P = NP$, considéré comme le problème fondamental du calcul mathématique ?

Exercice 2 : Machine de Turing

Démontrez que $P_n = 2^n + 2^{n-1}$ est multiple de 3, pour $n > 0$.

Soient $i_1, i_2, \dots, i_n$ des entiers supérieurs à 0, en déduire que la somme $P_{i_1} + \dots + P_{i_j} + P_{i_k} + \dots + P_{i_n}$ est multiple de 3. Pour l'exercice, on supposera que $abs(i_j - i_k) \geq 2$

Exemple : $P_8 + P_3 = (2^8 + 2^7) + (2^3 + 2^2)$ est multiple de 3.

Exemple : $P_{56} + P_{30} + P_4 = (2^{57} + 2^{56}) + (2^{30} + 2^{29}) + (2^4 + 2^3)$ est multiple de 3.

On s'intéresse à ces nombres, multiples de 3, qui peuvent s'écrire sous cette forme de somme de puissances de 2 (par paire), déterminez une machine de Turing qui décide l'ensemble de ces nombres binaires multiples de 3. Vous déterminerez les états, leur rôle et les transitions de votre machine.

Exercice 3 : Matrices creuses

On appelle une matrice "creuse", une matrice pour laquelle la plupart (90%) des éléments a une valeur nulle. Ainsi, pour la représenter, on propose ces deux solutions :

1) La matrice est représentée par un tableau à deux dimensions dont les cases contiennent des entiers.

2) La matrice est représentée par un tableau à une seule dimension, mais constituée d'une structure de trois entiers ; un triplet d'entiers (l, c, v) correspondant au numéro de la ligne (l), de la colonne (c) et de la valeur (v) des éléments non nuls de la matrice.

Question 1 :

Proposer une matrice creuse de dimension 10, représentée par un tableau à deux dimensions et par un tableau à une dimension, comme défini précédemment.

Question 2 :

On souhaite, pour une matrice de dimension $n > 1$, calculer la somme des différents éléments qui la composent. On demande d'écrire, pour chaque représentation, un algorithme permettant de retourner cette somme et donner, en le justifiant,

1. leur complexité spatiale (espace mémoire utilisé, c'est-à-dire le nombre d'entiers stockés) et

2. leur complexité temporelle (nombre d'opérations effectuées)

Question 3 :

Que peut-on conclure si l'on devait comparer les deux algorithmes précédents ? Prouver qu'il existe une valeur spécifique du nombre d'éléments non nuls à partir de laquelle une méthode l'emporte sur l'autre.

Exercice 4 : Complexité

Question 1 :

Proposez une formulation verbale et mathématique de la complexité de chacun de ces programmes P en fonction de n :

- Programme en général inutilisable, sauf cas très spécifique où n est vraiment petit
- Programme qui devient 2 fois plus lent quand n est doublé.
- Programme qui devient seulement légèrement plus lent quand n croît.
- Programme dont la complexité est typique du tri par insertion
- Programme utilisable, qu'avec de petites valeurs
- Programme dont la complexité est typique pour les algorithmes de type « diviser pour régner ». Chaque fois que n est doublé, la complexité est un peu plus que doublée

Question 2 :

$/$ désigne la division entière et le symbole $\%$ reste de la division entière. Par exemple :

$$5/2=2$$

$$5\%2=1$$

Démontrez ce que fait la fonction *mystere* suivante ; justifiez la complexité de cette fonction en nombre d'additions

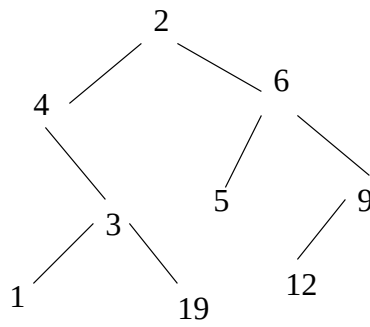
```
int mystere(int n){
    if (n == 0) return 0;
    return mystere(n-1) + (2*n) - 1
}
```

Justifiez, la complexité de cette fonction *mystere2* en nombre d'additions

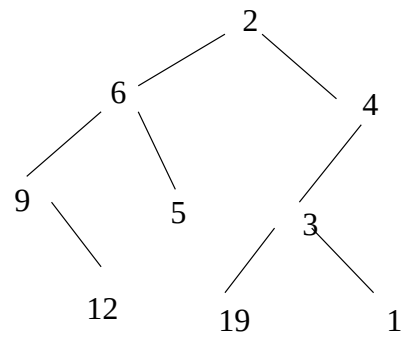
```
int mystere2(int n){
    if (n == 0) return 0;
    if (n == 1) return 1;
    return mystere(n) + mystere2(n/2)
}
```

Question 3 :

On veut calculer x^n , avec $n = 13$. En faisant aussi peu de multiplications que possible, proposez un algorithme le plus efficace possible, et trouvez le nombre minimum de multiplications. En déduire la complexité de votre algorithme.



arbre A



arbre miroir de A

FIGURE 1 – Le miroir d'un arbre

Question 4 :

Le miroir d'un arbre binaire (pas forcément de recherche) est l'arbre obtenu en inversant les branches droites et les branches gauches d'un arbre comme illustré par la figure 1.

Proposer une fonction qui prend en argument un arbre et le transforme en son miroir.

Donner, en le justifiant, la complexité en nombre d'appels récurifs en moyenne et dans le pire des cas de cette fonction.